

Representation-Aware Modularity with a Distance-Calibrated Null Model

CIKM 2026

Taejoon Han

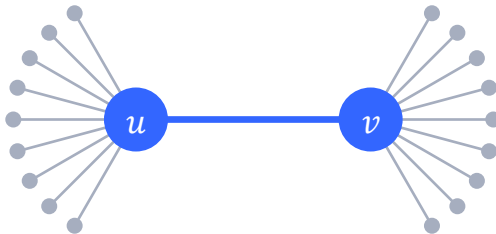
taejoon.han@unist.ac.kr

Department of Computer Science and Engineering
Ulsan National Institute of Science and Technology

Introduction

- **Community detection** identifies groups of nodes that are more densely connected to each other than to the rest of the graph.
- Among many formulations, **modularity** remains one of the most widely adopted objectives.
 - It compares the **observed** fraction of intra-community edges with the fraction **expected** under a **null model**.

$$Q(C) = \frac{1}{2m} \sum_{C \in \mathcal{C}} \sum_{u,v \in C} \left(\underbrace{A_{uv}}_{\text{observed}} - \underbrace{\gamma \frac{k_u k_v}{2m}}_{\text{expected (degree-based null model)}} \right)$$



observed $A_{uv} = 1$
expected $k_u k_v / 2m \approx 0.91$
→ surplus **+0.09** (expected anyway)

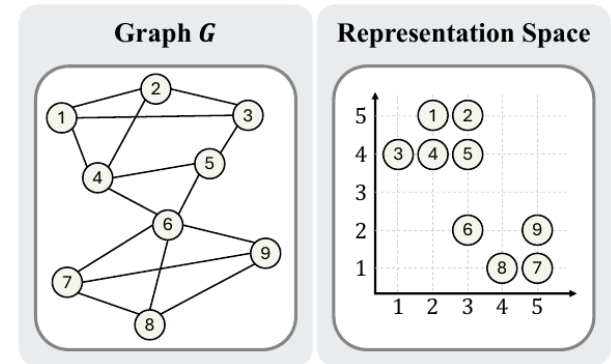
expected (degree-based null model)



observed $A_{uv} = 1$
expected $k_u k_v / 2m \approx 0.000002$
→ surplus $\approx$ **+1.00** (strong evidence)

Introduction

- Recent graph representation learning makes **node representations** a common input to mining tasks.
 - GNNs, random-walk embeddings, spectral methods, external features
 - Capture latent structural similarity, semantic proximity and higher-order patterns **beyond raw adjacency**.



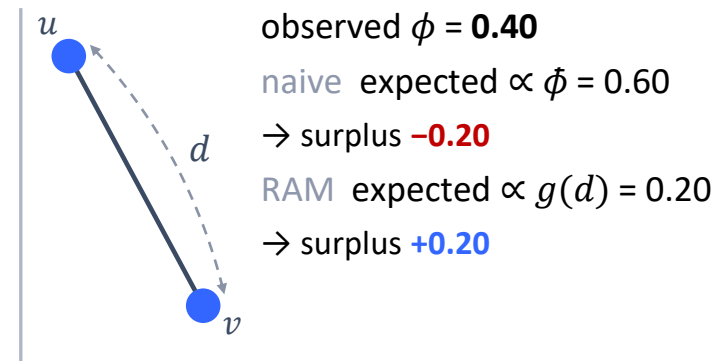
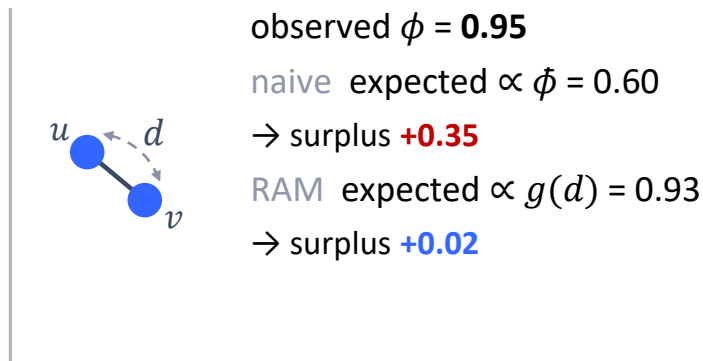
Q. How can modularity preserve its observed-minus-expected semantics when the observed affinity is representation-dependent?

Motivation: Null-Model Mismatch

- A **naive** extension replaces adjacency with representation-induced affinity but keeps the degree-based expected term:

$$Q_{\text{naive}}(C) = \frac{1}{2m} \sum_{C \in \mathcal{C}} \sum_{u,v \in C} \left[\underbrace{A_{uv} \cdot \phi(z_u, z_v)}_{\text{representation affinity}} - \underbrace{\frac{k_u k_v}{2m} \bar{\phi}}_{\text{global average affinity}} \right]$$

- Null-model mismatch**: the objective may reward affinity that is already expected from representation proximity.



$$\gamma \cdot k_u k_v / 2m \cdot \bar{\phi}$$

global average affinity



$$\gamma \cdot k_u k_v / 2m \cdot g(d_{uv})$$

expected affinity at distance $d_{uv} = \|z_u - z_v\|$

Representation-Aware Modularity

- $G = (V, E)$, $n = |V|$, $m = |E|$, $N(u)$: neighbours, $k_u = |N(u)|$
- Node representation $z_v \in \mathbb{R}^d$, $\mathcal{Z} = \{z_v\}_{v \in V}$
 - Treated as **input** — from GNNs, random walks, spectral methods or external features.

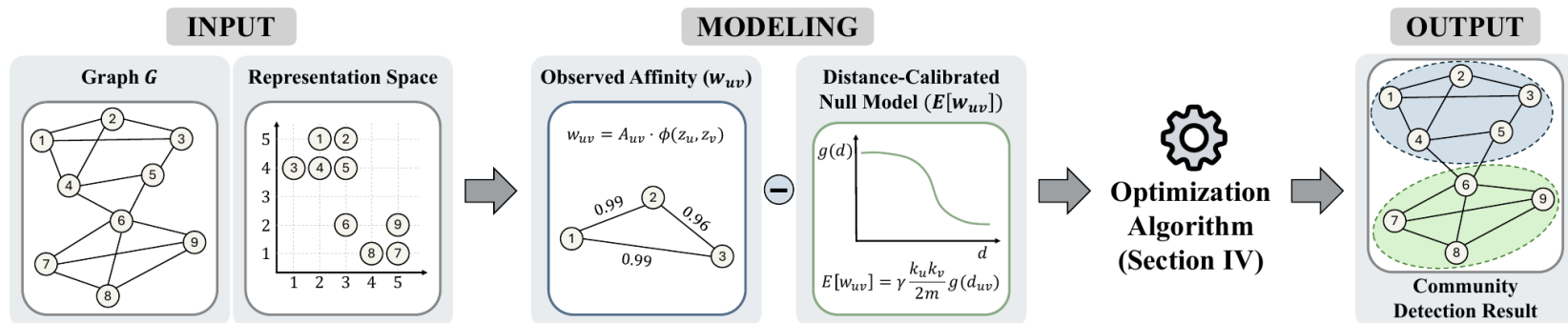
DEFINITION 1 (REPRESENTATION-AWARE MODULARITY (RAM)).
Given a graph $G = (V, E)$ with node representations $\mathcal{Z}$ and a partition $C = \{C_1, \dots, C_k\}$ of V , the representation-aware modularity is defined as

$$Q_{\text{RA}}(C) = \frac{1}{2m_w} \sum_{C \in \mathcal{C}} \sum_{u,v \in C} \left(\underline{w_{uv}} - \gamma \frac{k_u k_v}{2m} \underline{g(d_{uv})} \right)$$

- Rewards within-community affinity **only when it exceeds** what is expected from both **node degrees** and **representation distance**.
- Naive vs. RAM: representations enter **both** terms
 - **observed**: define the affinity $w_{uv} = A_{uv} \phi(z_u, z_v)$
 - **expected**: calibrate the baseline with $g(d_{uv})$ instead of the global ϕ

Problem Definition

PROBLEM DEFINITION 1 (REPRESENTATION-AWARE MODULARITY MAXIMISATION). Given an undirected graph $G = (V, E)$ and node representations $\mathcal{Z}$, the objective is to find an optimal partition $C^* = \{C_1, \dots, C_k\}$ of V such that $C^* = \arg \max_C Q_{RA}(C)$. The partition must satisfy $C_i \cap C_j = \emptyset$ for $i \neq j$ and $\bigcup_{i=1}^k C_i = V$.



- **Input:** graph G + node representations $\mathcal{Z}$
- **Modeling:** **observed affinity** w_{uv} on edges $\ominus$ **distance-calibrated null model** $g(d_{uv})$
- **Optimisation:** **BGD** (divisive search) / **MGA** (multi-level greedy, Louvain-style)
- **Output:** a community maximising RAM

Observed Term: Edge Affinity

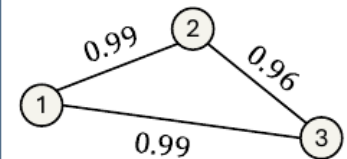
- Edge existence alone ignores **geometric proximity** between endpoints in the representation space.
- Representation-induced affinity on **observed edges**:

$$w_{uv} = A_{uv} \cdot \phi(z_u, z_v)$$

- $\phi(\cdot, \cdot)$: non-negative similarity (default: rescaled cosine $(1 + \cos)/2$)
- $w_{uv} = 0$ for $(u, v) \notin E \rightarrow$ affinity stays **graph-supported**
 - No node pair looks connected solely because it is close in the representation space.
- Total affinity mass $2m_w = \sum w_{uv}$ replaces the structural edge mass $2m$.

Observed Affinity (w_{uv})

$$w_{uv} = A_{uv} \cdot \phi(z_u, z_v)$$



Distance-Calibrated Null Model

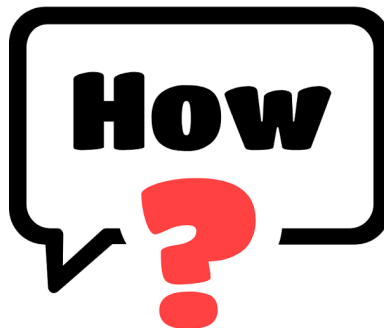
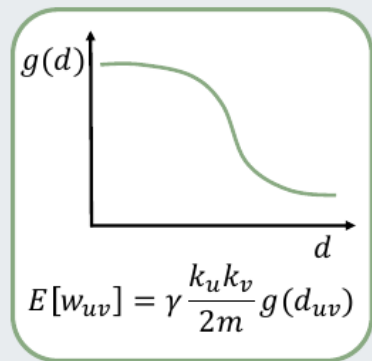
- **Why?** w_{uv} depends on geometry $\rightarrow$ a degree-only null model cannot capture geometry-induced variation.
- Expected affinity = **degree term** $\times$ **calibration term**

$$\mathbb{E}[w_{uv}] = \gamma \cdot k_u k_v / 2m \cdot g(d_{uv})$$

- **degree term**: as in classical modularity
- **calibration**: expected similarity at representation distance $d_{uv} = \|z_u - z_v\|$

$$g(d_{uv}) = \mathbb{E}[\phi(z_u, z_v) \mid \|z_u - z_v\| = d_{uv}, u \neq v]$$

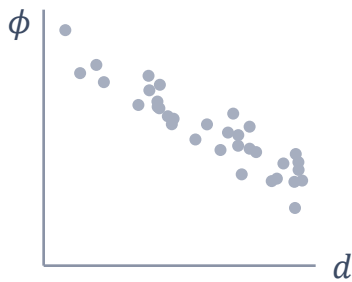
**Distance-Calibrated
Null Model ($\mathbb{E}[w_{uv}]$)**



Distance-Calibrated Null Model

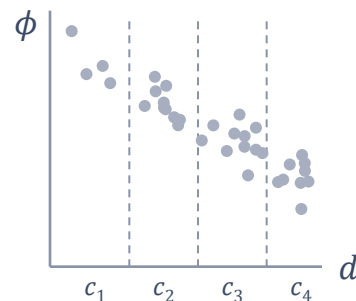
- **Why estimate?** Exact $g(d)$ needs the distribution of ϕ at every distance, which is **intractable** → approximate it from **sampled node pairs**.

① Sample pairs



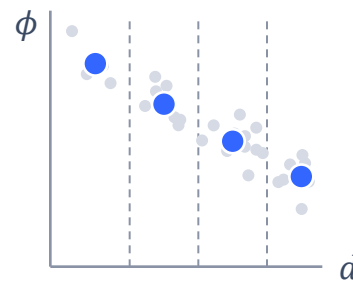
Draw pairs S uniformly ($u \neq v$); record d_{uv} and $\phi(z_u, z_v)$

② Bin distances



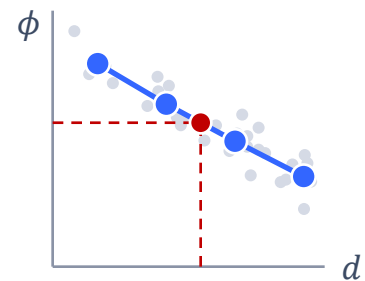
Split the distance range into L bins with centres c_ℓ

③ Average per bin



$\hat{g}_\ell$ = mean of ϕ over the pairs in bin ℓ

④ Interpolate



Connect adjacent $\hat{g}_\ell$ linearly → $\hat{g}(d)$ for any distance

- **In practice:** sample 30% of all pairs (capped at 5M); bins come from k -means on the distances, bin count chosen by silhouette score.

Algorithms

- Maximising RAM is challenging: combinatorial hardness + a null term that depends on **dense representation-distance interactions**.

BGD

Betweenness-Guided Divisive Search

- Top-down, Girvan–Newman style
- Repeatedly remove high weighted-betweenness edges, length $\ell_{uv} = 1/(\varepsilon + w_{uv})$
- Evaluate every split with RAM; keep the best partition
- **Costly** (repeated betweenness) → **reference optimiser**

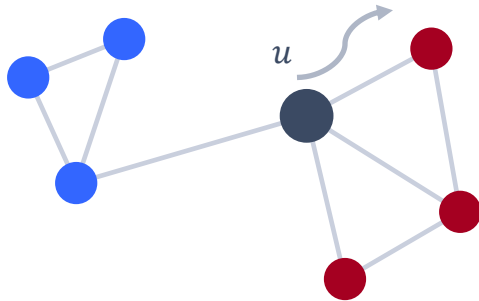
MGA

Multi-Level Greedy Aggregation

- Bottom-up, **Louvain-style**
- Local moves with RAM-specific gains + contraction into super-nodes
- **Centroid summaries** approximate the null term
- **Near-linear** → **main scalable optimiser**

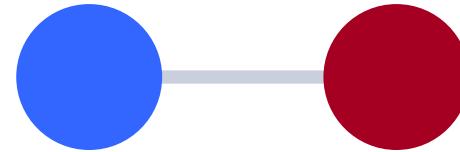
Recap: Louvain Algorithm

① Local moving



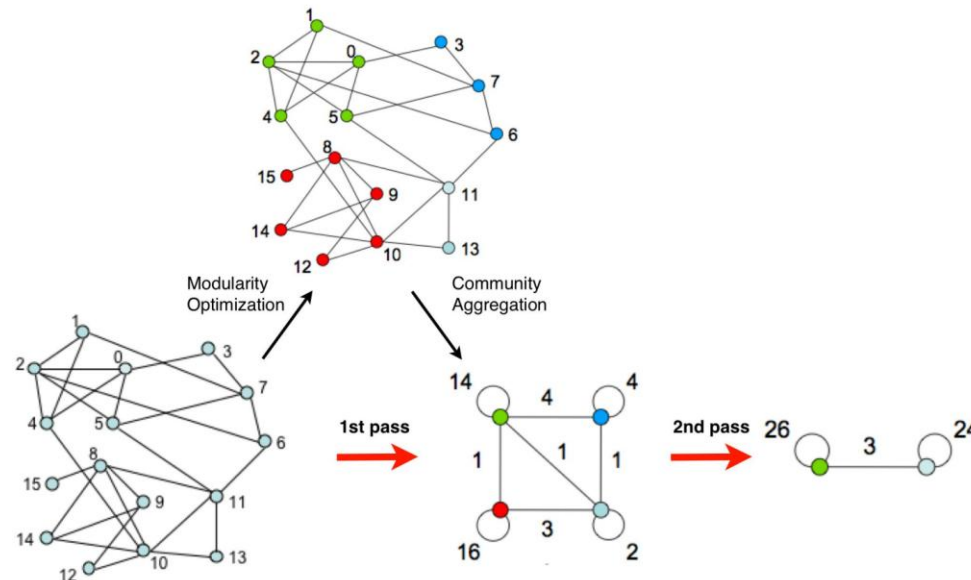
move u to the neighbouring community with max ΔQ

② Aggregation



communities $\rightarrow$ super-nodes, edge weights summed

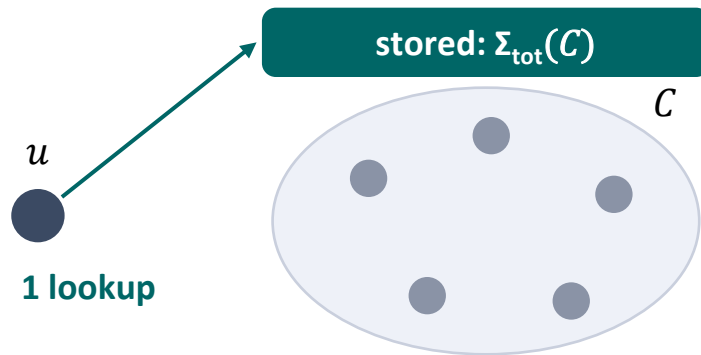
Repeat



Louvain Gain: $O(1)$ vs. RAM

Louvain: one stored number

$$\sum_{v \in C} k_u k_v / 2m = k_u \cdot \Sigma_{\text{tot}}(C) / 2m$$

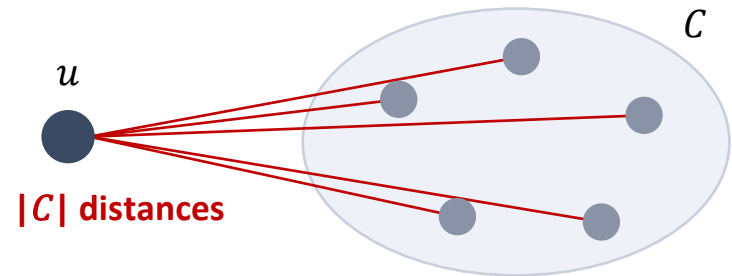


- k_u does not depend on $v \rightarrow$ **pull it out of the sum**
- Keep $\Sigma_{\text{tot}}(C)$ per community; on a move, add or subtract k_u

$O(1)$ per candidate community

RAM: one number is not enough

$$\sum_{v \in C} k_u k_v \cdot g(\|z_u - z_v\|) / 2m$$

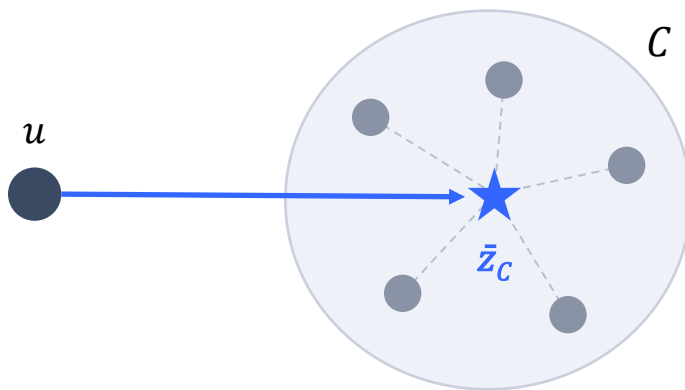


- g changes with **every pair** $(u, v) \rightarrow$ cannot be pulled out
- $\Sigma_{\text{tot}}(C)$ knows degrees, not how far each member is from u

$O(|C| \cdot d)$ per candidate community

MGA: Centroid Approximation

One distance instead of $|C|$



Members are summarised by their **centroid**; u measures one distance to it

1 Keep three summaries per community

$$K_C = \sum_{v \in C} k_v, \quad Z_C = \sum_{v \in C} z_v, \quad |C|$$

centroid $\bar{z}_C = Z_C / |C|$

2 Evaluate $\hat{g}$ once, at the centroid

$$\gamma \cdot k_u K_C / 2m \cdot \hat{g}(\|z_u - \bar{z}_C\|)$$

3 Update after moving u : $A \rightarrow B$

$$Z_A -= z_u, \quad K_A -= k_u, \quad |A| -= 1$$

$$Z_B += z_u, \quad K_B += k_u, \quad |B| += 1$$

Null term per candidate community: $O(|C| \cdot d) \rightarrow O(d)$

Experiment

Table 1: Real-world datasets

Type	Dataset	V	E	C	d_{avg}
Small-scale	Karate	34	78	2	4.5
	Cora	2,708	10,556	7	7.7
	Citeseer	3,327	9,104	6	5.4
	Amazon PC	13,752	491,722	10	35.7
Large-scale	DBLP*	317,080	1,049,866	13,477	6.6
	Amazon*	334,863	925,872	75,149	5.5
	YouTube*	1,134,890	2,987,624	8,385	5.2

* Dataset with overlapping ground-truth communities.

■ Baselines

- Topology: GN, GM, [Louvain](#), [Leiden](#), Infomap
- Embedding: k -means, Spectral, OPTICS, HDBSCAN (same embeddings as MGA)
- Deep: DMoN, DGCluster

■ Metrics

- Disjoint GT: NMI, ARI
- Overlapping GT: 1 – conductance, modularity

■ Setting

- ϕ = rescaled cosine, $\gamma = 1$, embeddings from AutoGL / DeepWalk

Experiment: Topology-based Baselines

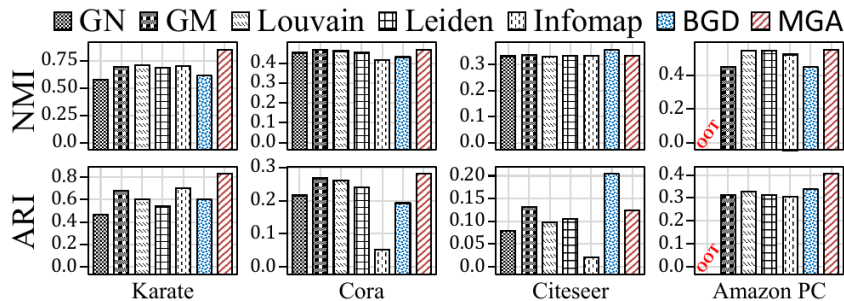


Figure 2: EQ1.1. Effectiveness on small-scale datasets

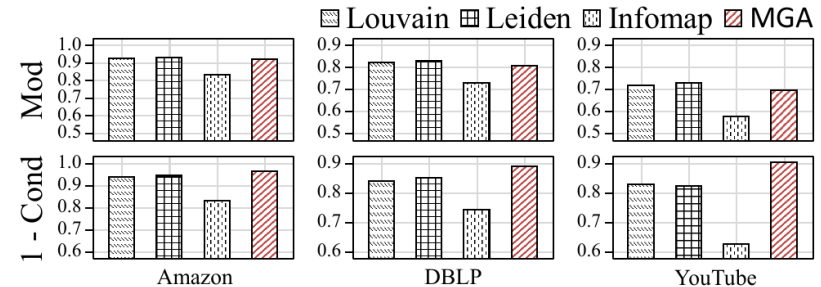


Figure 3: EQ1.2. Effectiveness on large-scale datasets (Mod: Modularity, 1 - Cond: 1 - conductance).

- **Small-scale:** MGA improves NMI / ARI by **1.05×** / **1.10×** on average over topology-only baselines.
 - Karate: NMI 0.852 (+20.5%), ARI 0.829 (+18.0%) over the best baseline
- **Large-scale:** best 1 - conductance on all three datasets (YouTube 0.906, **+9.0%**).
 - Modularity remains competitive with Leiden (avg. gap 0.022) although MGA does not optimise it directly.

Experiment: Embedding & Deep Baselines

Table 3: EQ2. Performance comparison on large-scale datasets (AG+: baseline with AutoGL embeddings (same as MGA))

Dataset	1 - Conductance ($\uparrow$)					Modularity ($\uparrow$)					Time (s) ($\downarrow$)				
	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB	MGA	AG+Kms	AG+Spec	AG+OPTICS	AG+HDB
Amazon	0.9669	0.8925	OOT	0.1882	0.3466	0.9214	0.3725	OOT	0.2269	0.4325	6.7645	284.2898	OOT	12408.9564	19545.6532
DBLP	0.8915	0.7558	OOT	0.1320	0.2728	0.8091	0.6808	OOT	0.1540	0.2862	4.6996	129.0685	OOT	5664.4762	6210.7943
YouTube	0.9060	0.6857	OOT	OOT	OOT	0.6969	0.0823	OOT	OOT	OOT	20.0166	685.6667	OOT	OOT	OOT

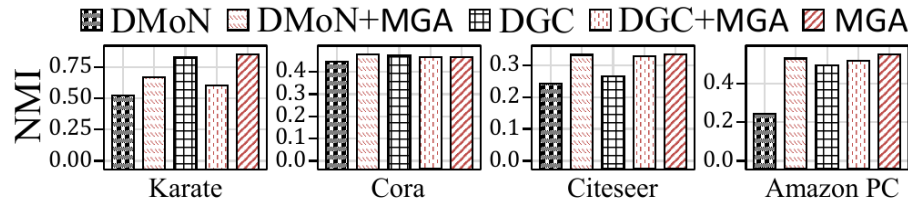


Figure 5: EQ3. Deep learning-based baselines

- **EQ2** (same embeddings): MGA is best in 1 – cond. and modularity on all large graphs; **>34×** faster than *k*-means, OPTICS/HDBSCAN often OOT.
- **EQ3**: DMoN+MGA improves DMoN NMI by **1.48×** (Amazon PC 0.243 → 0.530).
 - MGA acts as a representation-aware optimisation layer.

Experiment: Ablation (Distance Calibration)

Table 4: EQ5. Ablation variants of RAM

Variant	Observed term	Null term
RAM w/o deg	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \hat{g}(d_{uv})$
RAM w/o $\hat{g}$	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \frac{k_u k_v}{2m}$
RAM w $\bar{\phi}$	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \frac{k_u k_v}{2m} \bar{\phi}$
RAM w/o ϕ	A_{uv}	$\gamma \frac{k_u k_v}{2m} \hat{g}(d_{uv})$
RAM (full)	$A_{uv} \cdot \phi(z_u, z_v)$	$\gamma \frac{k_u k_v}{2m} \hat{g}(d_{uv})$

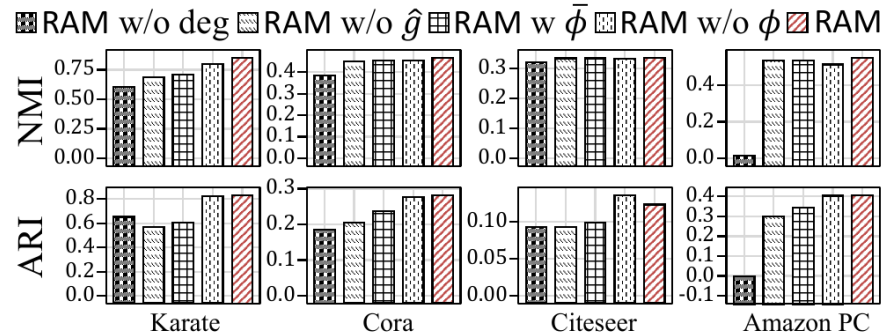


Figure 7: EQ5. Distance calibration

- **Full RAM** achieves the highest NMI and ARI in almost all cases.
- w/o deg: sharpest drop (Amazon PC NMI = 0.013) → **degree heterogeneity** matters.
- w/o $\hat{g}$ and w $\bar{\phi}$ (naive) are consistently worse → **distance calibration** matters.
- **Both degree correction and distance calibration are essential.**

Experiment: Efficiency & Scalability

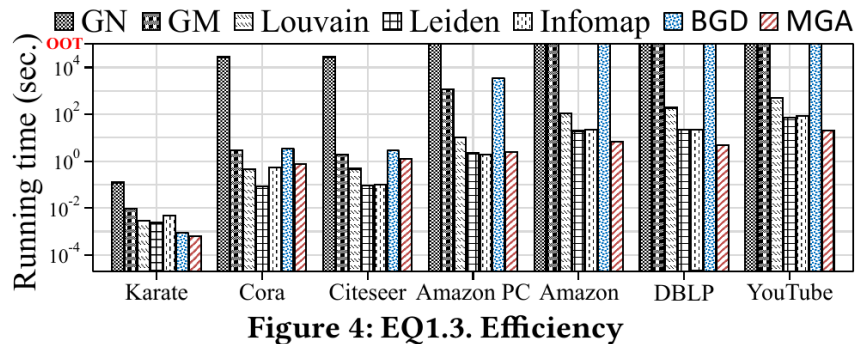


Figure 4: EQ1.3. Efficiency

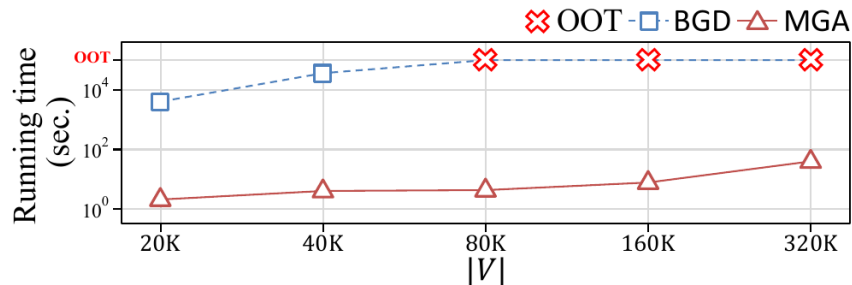


Figure 9: EQ8. Scalability

- **EQ1.3:** on large graphs MGA runs **3.69×** faster than Leiden on average.
 - Small graphs: precomputing $\hat{g}$ and w_{uv} takes >98% of time; optimisation < 0.02 s.
- **EQ8:** $|V| \times 16$ (20K $\rightarrow$ 320K) $\rightarrow$ running time only $\approx$ **19** (near-linear).
 - BGD takes >10 h at 40K and exceeds 24 h beyond.



Thank you